Lamhot Siagian

Top 50 PyTorch Interview Q&A

A technical study guide for machine learning engineers

Concepts, implementation patterns, debugging habits, and interview-ready explanations

---

Interview Prep Edition

2026 - AI Engineering Insider - interviewprep.cafe

Contents

Chapter 1: PyTorch Fundamentals & Tensors **4**
Q1: What is a PyTorch Tensor and how does it differ from a NumPy array? 4
Q2: Explain tensor creation methods: zeros, ones, rand, randn, arange, linspace. 5
Q3: How do tensor reshaping operations work? Contrast view, reshape, squeeze, unsqueeze. . 5
Q4: What are broadcasting rules in PyTorch and how can they cause silent bugs? 6
Q5: What is torch.einsum and when should you reach for it? 7

Chapter 2: Autograd & Computational Graphs **9**
Q6: How does PyTorch's automatic differentiation (autograd) work? 9
Q7: What is torch.no_grad() vs tensor.detach()? When do you use each? 9
Q8: What causes the in-place operation on a tensor required for gradient computation error? . 10
Q9: How do you implement a custom autograd Function? 11
Q10: What is gradient accumulation and why is it important for large-batch training? 12

Chapter 3: Neural Network Modules (nn.Module) **14**
Q11: What is the difference between nn.Parameter, register_buffer, and plain tensors? 14
Q12: How do forward hooks and backward hooks work in nn.Module? 14
Q13: How do you freeze model parameters for transfer learning? 15
Q14: What is weight sharing and how is it implemented in PyTorch? 16
Q15: How do nn.ModuleList, nn.ModuleDict, and nn.Sequential differ? 17

Chapter 4: Training Loop & Optimization **19**
Q16: Write a complete production-quality PyTorch training loop with validation. 19
Q17: Compare PyTorch optimizers: SGD, Adam, AdamW, RMSProp. 20
Q18: How do learning rate schedulers work? Compare StepLR, CosineAnnealingLR, OneCycleLR. 21
Q19: What loss functions does PyTorch provide and when should each be used? 22
Q20: What is mixed-precision training and how do you enable it? 23

Chapter 5: Data Loading & Preprocessing **24**
Q21: How do you implement a custom Dataset class in PyTorch? 24
Q22: What is an IterableDataset and when should you use it? 25
Q23: How do you write a custom collate_fn for variable-length sequences? 26
Q24: How do you profile DataLoader bottlenecks? 27
Q25: How do you handle class imbalance in PyTorch datasets? 28

Chapter 6: GPU Acceleration & CUDA **30**
Q26: How does PyTorch manage GPU memory, and how do you debug OOM errors? 30
Q27: What is pin_memory and why does it speed up host-to-device transfers? 30
Q28: How do CUDA streams work in PyTorch and when are they useful? 31
Q29: What is torch.compile() and how does it speed up training? 32
Q30: How do you profile a PyTorch model's GPU performance? 33

Chapter 7: Model Saving, Loading & Deployment **35**
Q31: What is the difference between saving state_dict vs saving the full model? 35
Q32: How do you export a PyTorch model to TorchScript? 35
Q33: How do you export a model to ONNX and run it with ONNX Runtime? 36
Q34: What is quantization in PyTorch and what are the different approaches? 37
Q35: How do you implement early stopping and model checkpointing? 38

Chapter 8: Advanced Architectures **40**
Q36: Implement a ResNet residual block from scratch. 40
Q37: Implement a multi-head self-attention layer from scratch. 40

Q38: How do you implement an LSTM and when would you use it vs a Transformer? 41
Q39: What is the difference between BatchNorm and LayerNorm? 42
Q40: Implement positional encoding for a Transformer. 42

Chapter 9: Distributed Training & Performance **44**
Q41: What is the difference between nn.DataParallel and DistributedDataParallel? 44
Q42: How does FSDP enable training of very large models? 45
Q43: How do you implement a DistributedSampler for DDP training? 46
Q44: What is pipeline parallelism vs tensor parallelism? 46
Q45: How do you use torchrun to configure a distributed training job? 47

Chapter 10: Debugging, Profiling & Best Practices **49**
Q46: How do you diagnose and fix vanishing/exploding gradients? 49
Q47: How do you track NaN values during training and identify their source? 49
Q48: How do you make PyTorch training fully reproducible? 50
Q49: How do you write unit tests for PyTorch neural network components? 51
Q50: What are common memory leaks in PyTorch training loops and how do you find them? . 52

Quick-reference study summary **54**

Chapter 1 PyTorch Fundamentals & Tensors

This chapter starts with tensors because almost every PyTorch interview does the same. If you are shaky on shapes, dtypes, or device placement, that weakness shows up again in training code, debugging, and deployment. The goal here is not to memorise APIs in isolation. It is to build enough intuition that you can read a tensor operation and immediately explain what it does, what shape it returns, and where it might fail.

Core ideas. Tensor creation, dtype handling, device placement, reshaping, broadcasting, and `einsum`. These are the building blocks behind nearly every training or inference pipeline.

What interviewers are really checking. Whether you can reason from shapes and data movement, not whether you can recite function names. Strong answers explain what a tensor operation returns, where it runs, and what can silently go wrong.

Common weak spots. Broadcasting that changes the result without throwing an error, accidental integer math, and CPU-GPU mismatches that only surface later in the workflow.

Study advice. For each example, say the input shape, output shape, dtype, and device out loud. If you can do that quickly, the rest of the book becomes much easier.

Question 1

What is a PyTorch Tensor and how does it differ from a NumPy array?

A **Tensor** is the fundamental data structure in PyTorch — a multi-dimensional array that can live on CPU or GPU and optionally track operations for automatic differentiation. Key differences from NumPy:

- **Device**: Tensors move to GPU via `.to('cuda')` or `.cuda()`.
- **Autograd**: Tensors with `requires_grad=True` build a computation graph and compute gradients.
- **Zero-copy bridge**: `torch.from_numpy()` shares memory with NumPy — no copy made.
- **Dtype system**: PyTorch supports `float16`, `bfloat16`, `float32`, `int8`, and more.

Python example

```python
import torch
import numpy as np

t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
n = np.array([[1.0, 2.0], [3.0, 4.0]])

# Zero-copy bridge -- shared memory
t_from_np = torch.from_numpy(n)
np_from_t  = t.numpy()            # CPU-only, no grad

# GPU movement
device = 'cuda' if torch.cuda.is_available() else 'cpu'
t_gpu  = t.to(device)
print(t_gpu.device)               # cuda:0  or  cpu

# Autograd
x = torch.tensor([2.0], requires_grad=True)
y = x ** 3
y.backward()
print(x.grad)                     # 3 * x^2 = 12.0
```

Question 2

Explain tensor creation methods: `zeros`, `ones`, `rand`, `randn`, `arange`, and `linspace`.

Each factory function serves a different initialization need. `zeros` and `ones` produce constant tensors; `rand` samples from Uniform$[0, 1)$; `randn` from $\mathcal{N}(0, 1)$; `arange` and `linspace` generate sequences. The `*_like` variants preserve device and dtype automatically.

Python example

```python
import torch

z  = torch.zeros(3, 4)            # all zeros  (3,4)
o  = torch.ones(2, 3)             # all ones
f  = torch.full((2, 3), 7.5)      # filled with 7.5

r  = torch.rand(3, 3)             # Uniform [0, 1)
rn = torch.randn(3, 3)            # Standard normal N(0,1)
ri = torch.randint(0, 10, (2,4))  # Integer in [0, 10)

a = torch.arange(0, 10, 2)        # [0, 2, 4, 6, 8]
l = torch.linspace(0, 1, 5)       # [0.0, 0.25, 0.5, 0.75, 1.0]

# Like-shape: preserves device & dtype
x      = torch.randn(3, 3).cuda()
z_like = torch.zeros_like(x)

torch.manual_seed(42)              # reproducibility
print(torch.randn(2, 2))
```

Question 3

How do tensor reshaping operations work? Contrast `view`, `reshape`, `squeeze`, and `unsqueeze`.

`view` is a zero-copy reshape but requires the tensor to be contiguous in memory. `reshape` is safer — it copies data when necessary. `squeeze` removes dimensions of size 1; `unsqueeze` inserts them. Shape bugs from these operations are among the most common sources of subtle errors in model code.

Python example

```python
import torch

t = torch.arange(24)              # shape: (24,)

# view: zero copy, contiguous required
t_v = t.view(4, 6)
t_v = t.view(2, 3, 4)

# reshape: copies if non-contiguous (safer)
t_r = t.reshape(3, 8)

# Contiguity issue
x = torch.randn(3, 4).T          # transpose -> non-contiguous
x_cont = x.contiguous()           # make contiguous
x_view = x_cont.view(12)          # now safe

# squeeze / unsqueeze
a = torch.randn(1, 3, 1, 5)      # (1, 3, 1, 5)
b = a.squeeze()                    # (3, 5)
c = a.squeeze(0)                   # (3, 1, 5)
d = b.unsqueeze(0)                 # (1, 3, 5)
d = b.unsqueeze(-1)                # (3, 5, 1)

f  = a.flatten()                   # (15,)
f2 = a.flatten(1)                  # (1, 15)
```

Question 4

What are broadcasting rules in PyTorch, and how can they cause silent bugs?

Broadcasting lets PyTorch operate on tensors with different shapes without copying data, aligning shapes from the right and stretching any dimension of size 1. When shapes are accidentally compatible, PyTorch silently broadcasts instead of raising an error — producing wrong results that are very hard to trace.

Python example

```python
import torch

# Valid broadcast: (3,) + (2,3) -> (2,3)
a = torch.tensor([1.0, 2.0, 3.0])   # (3,)
b = torch.ones(2, 3)                  # (2,3)
print((a + b).shape)                  # (2, 3)

# SILENT BUG: labels (3,) vs outputs (3,1) -> broadcasts to (3,3)
labels  = torch.tensor([1, 0, 1])          # (3,)
outputs = torch.tensor([[0.9],[0.1],[0.8]])# (3,1)
loss    = (outputs - labels) ** 2
print(loss.shape)    # (3,3)  -- WRONG: should be (3,)

# Fix: match shapes explicitly
labels_col = labels.float().unsqueeze(1)   # (3,1)
loss_fixed = (outputs - labels_col) ** 2
print(loss_fixed.shape)   # (3,1) correct
```

Pro Tip

Always verify shapes with `.shape` after arithmetic. Add `assert` statements in training code to catch broadcast surprises early — they cost nothing at runtime in normal execution.

Question 5

What is `torch.einsum` and when should you reach for it?

`einsum` expresses tensor contractions using Einstein summation notation. It handles matrix multiplications, dot products, outer products, batched operations, and attention score computations in a single readable line — making code intent immediately clear.

Python example

```python
import torch

a = torch.randn(3, 4);  b = torch.randn(4, 5)

# Matrix multiplication: 'ij,jk->ik'
c = torch.einsum('ij,jk->ik', a, b)         # (3,5)

# Dot product: 'i,i->'
x = torch.randn(5);  y = torch.randn(5)
dot = torch.einsum('i,i->', x, y)            # scalar

# Outer product: 'i,j->ij'
outer = torch.einsum('i,j->ij', x, y)        # (5,5)

# Batch matrix multiply: 'bij,bjk->bik'
A = torch.randn(8, 3, 4);  B = torch.randn(8, 4, 5)
C = torch.einsum('bij,bjk->bik', A, B)       # (8,3,5)

# Transformer attention scores: 'bqd,bkd->bqk'
Q = torch.randn(2, 10, 64)
K = torch.randn(2, 10, 64)
scores = torch.einsum('bqd,bkd->bqk', Q, K) # (2,10,10)

# Matrix trace: 'ii->'
M = torch.randn(4, 4)
trace = torch.einsum('ii->', M)
```

Chapter 2 Autograd & Computational Graphs

Autograd is where PyTorch stops being a tensor library and becomes a deep learning framework. This chapter focuses on what the engine is actually doing during forward and backward passes, and why that matters when gradients look wrong or memory starts climbing. These are the questions that separate someone who can use PyTorch from someone who can debug it under pressure.

Core ideas. Dynamic computation graphs, gradient flow, `requires_grad`, `no_grad`, `detach`, custom autograd logic, and gradient accumulation.

What interviewers are really checking. Whether you understand why gradients appear, disappear, or pile up. Good answers describe the mechanics of the graph instead of treating autograd like a black box.

Common weak spots. Forgetting to clear gradients, breaking the graph with in-place updates, and mixing inference shortcuts into training code.

Study advice. Trace one simple example by hand from forward pass to backward pass. Once you can explain where each gradient comes from, harder questions stop feeling abstract.

Question 6

How does PyTorch's automatic differentiation (autograd) work under the hood?

PyTorch uses **define-by-run** (dynamic) autodiff. Each operation on a tensor with `requires_grad=True` records a `grad_fn` on the output, building an acyclic computation graph. Calling `.backward()` traverses this graph in reverse, applying the chain rule at each node (reverse-mode AD).

Python example

```python
import torch

x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(4.0, requires_grad=True)

# z = (x+y)^2
z = x**2 + 2*x*y + y**2

print(z.grad_fn)              # <AddBackward0>
z.backward()
print(x.grad)                 # dz/dx = 2x+2y = 14.0
print(y.grad)                 # dz/dy = 2x+2y = 14.0

# Non-scalar backward needs explicit gradient argument
z2 = torch.randn(3, requires_grad=True) ** 2
z2.backward(torch.ones_like(z2))

print(x.is_leaf)              # True  (user created)
print(z.is_leaf)              # False (computed)
```

Question 7

What is `torch.no_grad()` vs `tensor.detach()`? When do you use each?

Both stop gradient tracking but at different levels. `torch.no_grad()` is a context manager that disables the gradient engine globally — used for inference to save memory and compute. `.detach()` returns a new tensor sharing the same data but removed from the graph — used when you need a tensor's value without backpropagating through it (e.g. target networks in RL, computing metrics).

Python example

```python
import torch

model_param = torch.randn(3, 3, requires_grad=True)

# torch.no_grad() -- disables engine globally
with torch.no_grad():
    out = model_param @ model_param.T
    print(out.requires_grad)      # False

# .detach() -- severs tensor from graph, shares memory
x     = torch.randn(3, requires_grad=True)
y     = x ** 2
y_val = y.detach()                # same data, no grad_fn
print(y_val.requires_grad)        # False

# DQN target network example
target = model_param.detach()     # stop gradients flowing here

# torch.inference_mode() -- strongest; tensors can never
# re-enter a graph.  Fastest for pure inference.
with torch.inference_mode():
    result = model_param.sum()
    print(result.requires_grad)   # False
```

Question 8

What causes the "in-place operation on a tensor required for gradient computation" error?

PyTorch needs the *original* values of tensors to compute gradients during backprop. In-place operations overwrite those values, corrupting the graph version counter and making the gradient computation incorrect or impossible. PyTorch raises a `RuntimeError` to protect you.

Python example

```python
import torch

# WRONG: in-place on a leaf tensor
x = torch.tensor([1.0, 2.0], requires_grad=True)
# x += 1   # RuntimeError: in-place on leaf!

# WRONG: in-place after grad_fn recorded
a = torch.randn(3, requires_grad=True)
b = a * 2            # grad_fn = MulBackward0
b += 1               # corrupts b's version
# b.sum().backward()  # RuntimeError

# RIGHT: out-of-place always creates a new tensor
b = b + 1            # safe: new tensor, graph intact
b.sum().backward()   # works fine

# In-place IS allowed on non-grad tensors (buffers)
buf = torch.zeros(3)
buf += 1.0           # fine -- no requires_grad
```

Question 9

How do you implement a custom autograd Function with forward and backward passes?

Subclass `torch.autograd.Function` and implement `forward` and `backward` as static methods. The canonical use case is a Straight-Through Estimator (STE) for discrete/non-differentiable operations such as quantization or binary activations.

Python example

```python
import torch
from torch.autograd import Function

class StraightThroughEstimator(Function):
    """
    Forward:  sign(x)  -- quantize to {-1, +1}
    Backward: pass gradient straight through (identity)
    """
    @staticmethod
    def forward(ctx, x):
        ctx.save_for_backward(x)
        return x.sign()

    @staticmethod
    def backward(ctx, grad_output):
        (x,) = ctx.saved_tensors
        grad = grad_output.clone()
        grad[x.abs() > 1] = 0          # clip outside [-1,1]
        return grad

ste = StraightThroughEstimator.apply
x   = torch.tensor([-1.5,-0.5,0.3,0.9,1.2], requires_grad=True)
y   = ste(x)
y.sum().backward()
print(x.grad)     # [0, 1, 1, 1, 0]

# Verify gradient numerically
from torch.autograd import gradcheck
x64 = torch.randn(4, dtype=torch.float64, requires_grad=True)
print(gradcheck(ste, (x64,), eps=1e-6))   # True
```

Question 10

What is gradient accumulation and why is it important for large-batch training?

When GPU memory cannot hold a large batch, you simulate it by accumulating gradients over multiple small forward/backward passes before calling `optimizer.step()`. Divide the loss by the accumulation steps so the effective gradient magnitude matches a true large batch.

Python example

```python
import torch, torch.nn as nn

model     = nn.Linear(512, 10)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
criterion = nn.CrossEntropyLoss()
ACCUM     = 4         # simulate batch_size * 4

optimizer.zero_grad()
for step, (x, y) in enumerate(dataloader):
    out  = model(x)
    loss = criterion(out, y) / ACCUM    # normalize!
    loss.backward()                      # grads accumulate

    if (step + 1) % ACCUM == 0:
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        optimizer.zero_grad()

# With AMP (mixed precision)
scaler = torch.cuda.amp.GradScaler()
for step, (x, y) in enumerate(dataloader):
    with torch.autocast(device_type='cuda', dtype=torch.float16):
        loss = criterion(model(x), y) / ACCUM
    scaler.scale(loss).backward()
    if (step + 1) % ACCUM == 0:
        scaler.unscale_(optimizer)
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer)
        scaler.update()
        optimizer.zero_grad()
```

Chapter 3 Neural Network Modules (nn.Module)

This chapter is about how PyTorch models are organised. Strong answers here sound less like documentation and more like engineering judgement: when to register a parameter, when to store a buffer, when to freeze layers, and how hooks help you inspect behaviour without rewriting the model. At the senior level, interviewers want to hear that you know how model code stays maintainable as it grows.

Core ideas. Parameters, buffers, hooks, weight sharing, freezing, and container classes such as `Sequential`, `ModuleList`, and `ModuleDict`.

What interviewers are really checking. Whether you understand how `nn.Module` handles registration, device movement, serialization, and what ends up inside the `state_dict`.

Common weak spots. Storing learnable tensors without registering them, confusing buffers with parameters, and using Python lists where a module container is required.

Study advice. Tie each concept to a model you have actually built. That is usually what separates a believable explanation from a memorized one.

Question 11

What is the difference between `nn.Parameter`, `register_buffer()`, and plain tensors?

	nn.Parameter	Buffer	Plain tensor
Learnable	Yes	No	No
state_dict	Yes	Yes	No
Moves with .to()	Yes	Yes	No
In optimizer	Yes	No	No

Python example

```python
import torch, torch.nn as nn

class DemoModule(nn.Module):
    def __init__(self):
        super().__init__()
        # Learnable, in optimizer, in state_dict
        self.weight = nn.Parameter(torch.randn(4, 4))
        # Non-learnable, in state_dict, moves with device
        self.register_buffer('running_mean', torch.zeros(4))
        # Not tracked at all -- lost on .to(), no grad
        self.plain = torch.randn(4)

    def forward(self, x):
        return x @ self.weight + self.running_mean

m = DemoModule()
print(m.state_dict().keys())  # weight, running_mean (no plain)
m.cuda()
print(m.weight.device)        # cuda:0
print(m.running_mean.device)  # cuda:0
print(m.plain.device)         # cpu   <-- gotcha!
```

Question 12

How do forward hooks and backward hooks work in `nn.Module`?

Hooks let you inspect or modify tensors flowing through a module without touching its source code. Forward hooks fire after `forward()`; backward hooks fire during `backward()`. They are essential for debugging, feature extraction, gradient surgery, and visualization. Always remove handles when done to avoid memory leaks.

Python example

```python
import torch, torch.nn as nn

model = nn.Sequential(nn.Linear(8,4), nn.ReLU(), nn.Linear(4,2))

# Forward hook: capture activations
activations = {}
def save_activation(name):
    def hook(module, inp, output):
        activations[name] = output.detach()
    return hook

handle = model[0].register_forward_hook(save_activation('l1'))
_ = model(torch.randn(2, 8))
print(activations['l1'].shape)  # (2, 4)
handle.remove()                  # always clean up!

# Backward hook: capture or modify gradients
grads = {}
def save_grad(name):
    def hook(grad):
        grads[name] = grad.detach()
    return hook

x = torch.randn(2, 8, requires_grad=True)
x.register_hook(save_grad('input'))
model(x).sum().backward()
print(grads['input'].shape)     # (2, 8)
```

Question 13

How do you freeze model parameters for transfer learning?

Set `requires_grad = False` on all parameters you want frozen, then replace the classification head with a new `nn.Linear` (which has `requires_grad=True` by default). Pass only the trainable parameters to the optimizer. Call `model.eval()` separately — it controls BatchNorm and Dropout behavior, independently of gradient freezing.

Python example

```python
import torch, torch.nn as nn
from torchvision.models import resnet50, ResNet50_Weights

model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

# Freeze entire backbone
for param in model.parameters():
    param.requires_grad = False

# Replace head (new layer has requires_grad=True)
model.fc = nn.Linear(2048, 10)

# Optimizer sees only trainable params
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-3
)

# Layer-wise learning rates
optimizer_lwlr = torch.optim.AdamW([
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(),     'lr': 1e-3},
], weight_decay=1e-2)

# Check trainable fraction
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
total     = sum(p.numel() for p in model.parameters())
print(f"Trainable: {trainable:,} / {total:,} ({100*trainable/total:.1f}%)")
```

Question 14

What is weight sharing in PyTorch and how is it implemented?

Weight sharing means multiple parts of a model reference the same `nn.Parameter` object. Gradients from all uses accumulate into that single parameter. Classic examples are tied encoder-decoder autoencoders and input-output embedding weight tying in language models.

Python example

```python
import torch, torch.nn as nn

class TiedAutoencoder(nn.Module):
    def __init__(self, input_dim, latent_dim):
        super().__init__()
        self.W = nn.Parameter(torch.randn(latent_dim, input_dim))
        self.enc_bias = nn.Parameter(torch.zeros(latent_dim))
        self.dec_bias = nn.Parameter(torch.zeros(input_dim))

    def encode(self, x):
        return torch.relu(x @ self.W.T + self.enc_bias)

    def decode(self, z):
        return z @ self.W + self.dec_bias  # tied: uses same W

    def forward(self, x):
        return self.decode(self.encode(x))

model = TiedAutoencoder(784, 64)
params = sum(p.numel() for p in model.parameters())
print(f"Parameters: {params:,}")  # 784*64 + 64 + 784 = 51040
```

Question 15

How do `nn.ModuleList`, `nn.ModuleDict`, and `nn.Sequential` differ?

All three register submodules so that `parameters()` and `state_dict()` include them. `Sequential` auto-chains its layers in `forward()`. `ModuleList` gives indexed access for custom forward logic. `ModuleDict` gives named access for conditional routing. Never use a plain Python `list` or `dict` — PyTorch will not see those modules.

Python example

```python
import torch, torch.nn as nn

# Sequential -- auto-forward, ordered
seq = nn.Sequential(nn.Linear(64,128), nn.ReLU(),
                    nn.Dropout(0.2), nn.Linear(128,10))
out = seq(torch.randn(8, 64))      # fully automatic

# ModuleList -- manual forward, indexed
class ResStack(nn.Module):
    def __init__(self, n):
        super().__init__()
        self.layers = nn.ModuleList([nn.Linear(64,64) for _ in range(n)])

    def forward(self, x):
        for layer in self.layers:
            x = torch.relu(layer(x)) + x   # residual
        return x

# ModuleDict -- named, conditional routing
class MultiHead(nn.Module):
    def __init__(self):
        super().__init__()
        self.heads = nn.ModuleDict({
            'cls': nn.Linear(64, 10),
            'reg': nn.Linear(64, 1),
        })

    def forward(self, x, task: str):
        return self.heads[task](x)
```

Chapter 4 Training Loop & Optimization

This is the chapter where practical PyTorch skill becomes obvious. A training loop can look clean on the surface and still hide stale gradients, bad scheduler timing, or unstable loss scaling. The point of this section is to walk through the moving parts in the order they appear in real work, so you can explain both the happy path and the failure cases.

Core ideas. Training-loop order, optimizer updates, schedulers, loss functions, gradient clipping, checkpoint logic, and mixed-precision training.

What interviewers are really checking. Whether you know the exact order of operations and can explain how correctness, numerical stability, and training speed fit together.

Common weak spots. Stepping the scheduler at the wrong moment, clipping gradients after the optimizer step, and using automatic mixed precision without understanding gradient scaling.

Study advice. Walk through a full loop from batch load to optimizer update without looking at the code. If you can narrate the order cleanly, you are in good shape.

Question 16

Write a complete, production-quality PyTorch training loop with validation.

A production training loop separates concerns into per-epoch functions, uses `set_to_none=True` for faster grad zeroing, applies gradient clipping, integrates mixed-precision via `GradScaler`, and saves the best checkpoint based on validation loss.

Python example

```python
import torch, torch.nn as nn

def train_epoch(model, loader, optimizer, criterion, device, scaler):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        optimizer.zero_grad(set_to_none=True)
        with torch.autocast(device_type='cuda', dtype=torch.float16):
            out  = model(x)
            loss = criterion(out, y)
        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(optimizer); scaler.update()
        total_loss += loss.item() * x.size(0)
        correct    += (out.argmax(1) == y).sum().item()
        total      += x.size(0)
    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        out   = model(x)
        loss  = criterion(out, y)
        total_loss += loss.item() * x.size(0)
        correct    += (out.argmax(1) == y).sum().item()
        total      += x.size(0)
    return total_loss / total, correct / total

device    = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model     = MyModel().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-2)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=50)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
scaler    = torch.cuda.amp.GradScaler()
best_val  = float('inf')

for epoch in range(1, 51):
    tr_loss, tr_acc = train_epoch(model, train_loader, optimizer, criterion, device, scaler)
    vl_loss, vl_acc = evaluate(model, val_loader, criterion, device)
    scheduler.step()
    if vl_loss < best_val:
        best_val = vl_loss
        torch.save(model.state_dict(), 'best.pt')
    print(f"Ep {epoch:03d} | Train {tr_loss:.4f}/{tr_acc:.3f} | Val {vl_loss:.4f}/{vl_acc:.3f}")
```

Question 17

Compare PyTorch optimizers: SGD, Adam, AdamW, and RMSProp.

The critical distinction is **Adam vs AdamW**: in Adam, weight decay is scaled by the adaptive step size (incorrect regularisation). AdamW decouples weight decay from the adaptive update, giving proper L2 regularisation. Always prefer AdamW for modern deep networks.

Optimizer	Adaptive	Momentum	Best for
SGD	No	Yes	Vision, fine-tuning, SGD+momentum
Adam	Yes	Yes	General use
AdamW	Yes	Yes	Transformers, large models
RMSProp	Yes	Optional	RNNs, RL

Python example

```python
import torch, torch.nn as nn
model = nn.Linear(128, 10)

sgd   = torch.optim.SGD(model.parameters(),
          lr=0.01, momentum=0.9, weight_decay=1e-4, nesterov=True)

adam  = torch.optim.Adam(model.parameters(),
          lr=1e-3, betas=(0.9, 0.999), eps=1e-8)

# AdamW: weight decay decoupled from adaptive step
adamw = torch.optim.AdamW(model.parameters(),
          lr=3e-4, betas=(0.9, 0.999), weight_decay=1e-2)

rms   = torch.optim.RMSprop(model.parameters(),
          lr=1e-3, alpha=0.99, eps=1e-8)

# AdamW formula (correct):
# theta -= lr * m / (sqrt(v) + eps)
# theta -= lr * wd * theta    <-- separate, unscaled decay
```

Question 18

How do learning rate schedulers work? Compare `StepLR`, `CosineAnnealingLR`, and `OneCycleLR`.

`StepLR` multiplies LR by `gamma` every `step_size` epochs — simple, but creates sharp discontinuities. `CosineAnnealingLR` smoothly anneals from max to `eta_min` — standard for modern training. `OneCycleLR` (per-batch) implements Leslie Smith's super-convergence: linear warmup to `max_lr` then cosine decay. Use `SequentialLR` to combine warmup with any other scheduler.

Python example

```python
import torch, torch.nn as nn
optimizer = torch.optim.SGD(nn.Linear(10,1).parameters(), lr=0.1)

step_sched = torch.optim.lr_scheduler.StepLR(
    optimizer, step_size=10, gamma=0.1)

cos_sched  = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=50, eta_min=1e-6)

one_sched  = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=0.1,
    steps_per_epoch=len(train_loader), epochs=30,
    pct_start=0.3, anneal_strategy='cos')

# Warmup + cosine with SequentialLR
warmup = torch.optim.lr_scheduler.LinearLR(
    optimizer, start_factor=0.1, total_iters=5)
main   = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=45)
sched  = torch.optim.lr_scheduler.SequentialLR(
    optimizer, schedulers=[warmup, main], milestones=[5])

# Per-epoch schedulers: call after optimizer.step()
for epoch in range(50):
    train_one_epoch(...)
    cos_sched.step()

# OneCycleLR: call per batch
for x, y in train_loader:
    optimizer.step(); one_sched.step()
```

Question 19

What loss functions does PyTorch provide and when should each be used?

Loss	Task	Notes
CrossEntropyLoss	Multi-class	Accepts raw logits
BCEWithLogitsLoss	Binary	Numerically stable
MSELoss	Regression	Sensitive to outliers
L1Loss	Regression	Robust to outliers
HuberLoss	Regression	L1/L2 hybrid
NLLLoss	Multi-class	Needs log-softmax first

Python example

```python
import torch, torch.nn as nn

ce     = nn.CrossEntropyLoss(label_smoothing=0.1)
bce    = nn.BCEWithLogitsLoss()
mse    = nn.MSELoss()
huber  = nn.HuberLoss(delta=1.0)

# Class imbalance: weight rare classes more
w  = torch.tensor([1.0, 5.0, 3.0]).cuda()
ce_w = nn.CrossEntropyLoss(weight=w)

# Custom Focal Loss (reduces easy-example weight)
class FocalLoss(nn.Module):
    def __init__(self, alpha=1.0, gamma=2.0):
        super().__init__()
        self.alpha, self.gamma = alpha, gamma

    def forward(self, inputs, targets):
        bce  = nn.functional.binary_cross_entropy_with_logits(
                 inputs, targets.float(), reduction='none')
        p_t  = torch.exp(-bce)
        return (self.alpha * (1 - p_t) ** self.gamma * bce).mean()
```

Question 20

What is mixed-precision training and how do you enable it in PyTorch?

Mixed precision uses `float16` (or `bfloat16`) for compute and `float32` for weight updates. This halves memory and speeds up Tensor Core operations. `GradScaler` prevents gradient underflow when using `float16`. On Ampere+ GPUs (A100, H100), prefer `bfloat16` which has the same exponent range as `float32` and needs no scaler.

Python example

```python
import torch, torch.nn as nn

model     = nn.Linear(1024, 1024).cuda()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
scaler    = torch.cuda.amp.GradScaler()

for x, y in dataloader:
    x, y = x.cuda(), y.cuda()
    optimizer.zero_grad(set_to_none=True)

    # autocast: ops inside run in fp16 automatically
    with torch.autocast(device_type='cuda', dtype=torch.float16):
        out  = model(x)
        loss = criterion(out, y)

    scaler.scale(loss).backward()
    scaler.unscale_(optimizer)
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()

# bfloat16 on Ampere+ -- no scaler needed
with torch.autocast(device_type='cuda', dtype=torch.bfloat16):
    out = model(x)   # stable without GradScaler
```

Chapter 5 Data Loading & Preprocessing

A model can only train as fast as data arrives. This chapter focuses on the side of PyTorch that quietly controls throughput: dataset design, worker configuration, batching strategy, and collation. Interviewers like these questions because they reveal whether you have worked on real pipelines or only on small notebook-sized datasets.

Core ideas. `Dataset`, `IterableDataset`, `DataLoader`, sampling strategies, custom `collate_fn`, and preprocessing design for larger workloads.

What interviewers are really checking. Whether you can connect elegant dataset code with real throughput constraints such as I/O, CPU transforms, batching cost, and host-to-device transfer time.

Common weak spots. Treating the dataloader as a detail, missing bottlenecks outside the model, and underestimating the effect of sampling or padding on training behavior.

Study advice. When you review an example, ask where the pipeline would slow down first in production. That question often leads to better answers than API memorization.

Question 21

How do you implement a custom `Dataset` class in PyTorch?

A map-style `Dataset` requires `__len__` and `__getitem__`. The `DataLoader` wraps it and handles batching, shuffling, and multi-process loading. Key performance flags: `num_workers`, `pin_memory`, `persistent_workers`, and `prefetch_factor`.

Python example

```python
import torch
from torch.utils.data import Dataset, DataLoader
from pathlib import Path
from PIL import Image
import torchvision.transforms as T

class ImageDataset(Dataset):
    def __init__(self, root, split='train', transform=None):
        self.root    = Path(root) / split
        self.classes = sorted(d.name for d in self.root.iterdir() if d.is_dir())
        self.class_to_idx = {c: i for i, c in enumerate(self.classes)}
        self.samples = [(p, self.class_to_idx[p.parent.name])
                        for cls_dir in self.root.iterdir()
                        for p in cls_dir.glob('*.jpg')]
        self.transform = transform or T.ToTensor()

    def __len__(self):   return len(self.samples)

    def __getitem__(self, idx):
        path, label = self.samples[idx]
        img = Image.open(path).convert('RGB')
        return self.transform(img), label

loader = DataLoader(
    ImageDataset('data', 'train',
      transform=T.Compose([T.RandomResizedCrop(224),
        T.RandomHorizontalFlip(), T.ToTensor(),
        T.Normalize([0.485,0.456,0.406],[0.229,0.224,0.225])])),
    batch_size=64, shuffle=True,
    num_workers=8,              # parallel workers
    pin_memory=True,            # faster CPU->GPU transfer
    persistent_workers=True,    # keep workers alive
    prefetch_factor=2,          # batches to prefetch
    drop_last=True,
)
```

Question 22

What is an `IterableDataset` and when should you use it over a `Dataset`?

Use `IterableDataset` when data is too large for RAM, arrives as a stream, or when random access is expensive (e.g. streaming from S3 or a database). It yields examples sequentially rather than by index. Always implement worker sharding inside `__iter__` using `get_worker_info()` — without it every worker produces the full dataset.

Python example

```python
import torch
from torch.utils.data import IterableDataset, DataLoader

class StreamingDataset(IterableDataset):
    def __init__(self, file_paths, seq_len=512):
        self.files, self.seq_len = file_paths, seq_len

    def __iter__(self):
        info = torch.utils.data.get_worker_info()
        files = (self.files[info.id::info.num_workers]
                 if info else self.files)

        for path in files:
            tokens = tokenize(open(path).read())
            for i in range(0, len(tokens) - self.seq_len, self.seq_len):
                chunk = tokens[i : i + self.seq_len]
                yield torch.tensor(chunk), torch.tensor(chunk[1:]+[0])

loader = DataLoader(
    StreamingDataset(['shard_0.txt', 'shard_1.txt']),
    batch_size=32, num_workers=4,
    # shuffle=False: iterable controls order
)
```

Question 23

How do you write a custom `collate_fn` for variable-length sequences?

The default collate function requires all tensors in a batch to have the same shape. For variable-length sequences, provide a custom `collate_fn` that pads shorter sequences to the longest in the batch using `pad_sequence`, and optionally sorts by length for efficient RNN packing.

Python example

```python
import torch
from torch.nn.utils.rnn import pad_sequence

def collate_variable_length(batch):
    sequences, labels = zip(*batch)

    # Sort descending by length (required for pack_padded_sequence)
    idx       = sorted(range(len(sequences)),
                       key=lambda i: len(sequences[i]), reverse=True)
    sequences = [sequences[i] for i in idx]
    labels    = [labels[i]    for i in idx]

    padded  = pad_sequence(sequences, batch_first=True, padding_value=0)
    lengths = torch.tensor([len(s) for s in sequences])
    return padded, lengths, torch.tensor(labels)

# Multi-label collate (ragged label lists)
def collate_multi_label(batch):
    images, label_lists = zip(*batch)
    images  = torch.stack(images)
    max_len = max(len(l) for l in label_lists)
    labels  = torch.full((len(batch), max_len), -1)
    for i, l in enumerate(label_lists):
        labels[i, :len(l)] = torch.tensor(l)
    return images, labels

loader = DataLoader(dataset, batch_size=32,
                    collate_fn=collate_variable_length)
```

Question 24

How do you profile DataLoader bottlenecks and optimize throughput?

Time batches explicitly with CUDA synchronization to isolate loading vs compute overhead. If the loader is the bottleneck, increase `num_workers`, enable `pin_memory` and `persistent_workers`, and consider moving heavy augmentations to the GPU using `torchvision.transforms.v2`.

Python example

```python
import torch, time

def benchmark_loader(loader, n=50):
    it = iter(loader)
    next(it)                          # warm up
    torch.cuda.synchronize()
    t0 = time.perf_counter()
    for i, batch in enumerate(it):
        if i >= n: break
        _ = batch[0].cuda(non_blocking=True)
    torch.cuda.synchronize()
    elapsed = time.perf_counter() - t0
    print(f"{n} batches: {elapsed:.2f}s ({n/elapsed:.1f}/s)")

# GPU-side augmentation (3-5x faster than CPU transforms)
import torchvision.transforms.v2 as Tv2
gpu_aug = Tv2.Compose([Tv2.RandomHorizontalFlip(),
                       Tv2.ColorJitter(0.4, 0.4, 0.4),
                       Tv2.RandomErasing(p=0.5)])

for x, y in loader:
    x = x.cuda()
    x = gpu_aug(x)     # all ops on GPU
```

Question 25

How do you handle class imbalance in PyTorch datasets?

Three complementary approaches: (1) `WeightedRandomSampler` to oversample rare classes at the batch level; (2) weighted loss to penalize misclassifying rare classes more; (3) Focal Loss to down-weight the easy majority-class examples during training.

Python example

```python
import torch, numpy as np
from torch.utils.data import WeightedRandomSampler

labels = [0]*900 + [1]*90 + [2]*10     # 9:1:0.1 imbalance
counts  = np.bincount(labels)
weights = 1.0 / counts                  # inverse frequency
sample_weights = torch.tensor([weights[l] for l in labels])

sampler = WeightedRandomSampler(
    weights=sample_weights, num_samples=len(labels),
    replacement=True)

loader = DataLoader(dataset, batch_size=32, sampler=sampler)

# Weighted cross-entropy loss
class_w = torch.tensor([1.0, 10.0, 100.0]).cuda()
ce_w    = torch.nn.CrossEntropyLoss(weight=class_w)

# Focal loss (gamma reduces easy-example contribution)
class FocalLoss(torch.nn.Module):
    def __init__(self, gamma=2):
        super().__init__(); self.gamma = gamma
    def forward(self, logits, targets):
        log_p  = torch.nn.functional.log_softmax(logits, -1)
        log_pt = log_p.gather(1, targets.view(-1,1)).squeeze()
        return (-(1 - log_pt.exp())**self.gamma * log_pt).mean()
```

Chapter 6 GPU Acceleration & CUDA

Moving a model to CUDA is easy. Getting consistent speedups without running into stalls, sync points, or out-of-memory crashes is harder. This chapter treats GPU work as a systems problem. It shows how memory moves, where overhead appears, and what to profile first when the model is slower than expected.

Core ideas. Device placement, pinned memory, asynchronous transfer, CUDA streams, compilation, profiling, and memory debugging.

What interviewers are really checking. Whether you know what actually overlaps on the GPU, what forces synchronization, and why utilization numbers can be misleading.

Common weak spots. Timing CUDA code without synchronization, blaming the model for a data pipeline bottleneck, and confusing allocated memory with reserved memory.

Study advice. Read this chapter with a systems mindset. Every speedup claim should come with a reason, a measurement method, and a note about what tradeoff it introduces.

Question 26

How does PyTorch manage GPU memory, and how do you debug OOM errors?

PyTorch uses a caching allocator: freed memory is returned to a pool rather than the OS, so `memory_reserved` ≥ `memory_allocated`. OOM errors usually come from storing computation graphs in lists, forgetting `.item()`, skipping `no_grad()` at eval time, or batch sizes that are simply too large.

Python example

```python
import torch

print(torch.cuda.memory_allocated() / 1e9, "GB")
print(torch.cuda.memory_reserved()  / 1e9, "GB")
print(torch.cuda.max_memory_allocated() / 1e9, "GB peak")

# Memory snapshot (PyTorch >= 2.0)
torch.cuda.memory._record_memory_history(max_entries=100_000)
# ... training steps ...
torch.cuda.memory._dump_snapshot('mem.pickle')

# WRONG: accumulates graph references
losses = []
for x, y in loader:
    loss = model(x).sum()
    losses.append(loss)          # holds live graph!

# RIGHT: extract Python scalar
losses.append(loss.item())       # no graph held

# Release fragmented cache
torch.cuda.empty_cache()

# Flash Attention: O(n) memory (no n^2 matrix materialised)
out = torch.nn.functional.scaled_dot_product_attention(
    Q, K, V, is_causal=True)
```

Question 27

What is `pin_memory` and why does it speed up host-to-device transfers?

Pinned (page-locked) memory cannot be swapped to disk, which allows the GPU DMA engine to transfer data directly from CPU RAM without an intermediate copy. Combined with `non_blocking=True` on the `.to()` call, the CPU–GPU transfer overlaps with Python execution, giving 2–3x throughput improvement on large batches.

Python example

```python
import torch
from torch.utils.data import DataLoader

loader = DataLoader(dataset, batch_size=128,
                    pin_memory=True, num_workers=4)

for x, y in loader:
    # non_blocking: transfer starts, CPU continues immediately
    x = x.to('cuda', non_blocking=True)
    y = y.to('cuda', non_blocking=True)
    # GPU kernel will wait for transfer automatically
    loss = criterion(model(x), y)

# Manual pinned tensor
t = torch.randn(1024, 1024).pin_memory()
print(t.is_pinned())            # True
t_gpu = t.to('cuda', non_blocking=True)

# CAUTION: only enable on GPU workloads.
# CPU-only training pays allocation cost with no benefit.
```

Question 28

How do CUDA streams work in PyTorch and when are they useful?

A CUDA stream is a queue of GPU operations that execute in order. Operations in different streams can run concurrently when hardware resources allow. The primary use case is overlapping data transfer with computation — while the GPU computes batch N, the DMA engine pre-loads batch $N + 1$.

Python example

```python
import torch

compute_stream  = torch.cuda.Stream()
transfer_stream = torch.cuda.Stream()

x_cpu = torch.randn(1024, 1024).pin_memory()
model = torch.nn.Linear(1024, 512).cuda()

with torch.cuda.stream(transfer_stream):
    x_gpu = x_cpu.to('cuda', non_blocking=True)

# Record event: compute stream waits for transfer
event = torch.cuda.Event()
transfer_stream.record_event(event)

with torch.cuda.stream(compute_stream):
    compute_stream.wait_event(event)
    out = model(x_gpu)

torch.cuda.synchronize()     # wait for all streams

# Double-buffering pattern: GPU computes N, CPU loads N+1
prev_x = None
for x_cpu, y in loader:
    if prev_x is not None:
        with torch.cuda.stream(compute_stream):
            out = model(prev_x)
    with torch.cuda.stream(transfer_stream):
        x_gpu = x_cpu.to('cuda', non_blocking=True)
    torch.cuda.synchronize()
    prev_x = x_gpu
```

Question 29

What is `torch.compile()` and how does it speed up training?

`torch.compile()` (PyTorch 2.0+) captures Python-level computation graphs via TorchDynamo and generates optimised Triton kernels via TorchInductor. It fuses operations, eliminates Python overhead, and unlocks hardware-specific optimisations — typically 1.5–3x speedup on modern GPUs with no model code changes.

Python example

```python
import torch, torch.nn as nn, time

model = nn.Transformer(d_model=512, nhead=8).cuda()

# Default backend: inductor (generates Triton kernels)
compiled = torch.compile(model)

# Modes: 'default' | 'reduce-overhead' | 'max-autotune'
fast_model = torch.compile(model, mode='max-autotune',
                           fullgraph=True)

# First forward triggers compilation (cold start)
x   = torch.randn(8, 32, 512).cuda()
out = compiled(x, x)       # compiles here

# Measure speedup
N = 100
start = time.perf_counter()
for _ in range(N): model(x, x)
torch.cuda.synchronize()
t_eager = time.perf_counter() - start

start = time.perf_counter()
for _ in range(N): compiled(x, x)
torch.cuda.synchronize()
t_compiled = time.perf_counter() - start

print(f"Speedup: {t_eager/t_compiled:.2f}x")
```

Question 30

How do you profile a PyTorch model's GPU performance?

Use the PyTorch Profiler to get CPU and CUDA timings per operator, with TensorBoard integration for visual inspection. Use `torch.cuda.Event` for lightweight micro-benchmarking of specific operations.

Python example

```python
import torch
from torch.profiler import profile, record_function, ProfilerActivity

model = MyModel().cuda()

with profile(
    activities=[ProfilerActivity.CPU, ProfilerActivity.CUDA],
    record_shapes=True, profile_memory=True,
    on_trace_ready=torch.profiler.tensorboard_trace_handler('./log'),
    schedule=torch.profiler.schedule(wait=1, warmup=1,
                                     active=3, repeat=1),
) as prof:
    for step, (x, y) in enumerate(loader):
        with record_function("forward"):  out = model(x)
        with record_function("backward"): out.sum().backward()
        prof.step()

print(prof.key_averages().table(sort_by="cuda_time_total",
                                row_limit=15))

# Lightweight: CUDA events for micro-benchmarking
start = torch.cuda.Event(enable_timing=True)
end   = torch.cuda.Event(enable_timing=True)
start.record()
out = model(x)
end.record()
torch.cuda.synchronize()
print(f"Forward: {start.elapsed_time(end):.2f} ms")
```

Chapter 7 Model Saving, Loading & Deployment

Training is not the finish line. At some point the model has to be saved, loaded somewhere else, and run predictably outside the original notebook or training job. This chapter is built around that handoff. It focuses on the choices that affect portability, reproducibility, latency, and how much pain you create for the next person who has to load the artifact.

Core ideas. `state_dict` usage, checkpoint design, resume logic, TorchScript, ONNX export, quantization, and deployment-facing model packaging.

What interviewers are really checking. Whether you can match the export path to the target environment and explain what must be saved for reliable recovery or serving.

Common weak spots. Saving too little to resume training, saving too much without structure, and treating every deployment format as interchangeable.

Study advice. Practice answering from a deployment scenario: mobile, server inference, research handoff, or training recovery. Context usually makes the tradeoff easier to explain.

Question 31

What is the difference between saving `model.state_dict()` and saving the entire model?

Always save `state_dict` — it serialises only parameters and buffers as plain tensors. Full model save uses Python pickle and includes the class definition, so it breaks if the class is renamed or moved. For training checkpoints, save optimizer, scheduler, and epoch alongside the weights.

Python example

```python
import torch, torch.nn as nn

model = nn.Linear(64, 10)

# Recommended: state_dict only
torch.save(model.state_dict(), 'weights.pt')
model2 = nn.Linear(64, 10)
model2.load_state_dict(torch.load('weights.pt', map_location='cpu'))

# Full checkpoint for resuming training
def save_ckpt(model, optimizer, scheduler, epoch, loss, path):
    torch.save({'epoch': epoch, 'loss': loss,
                'model': model.state_dict(),
                'optim': optimizer.state_dict(),
                'sched': scheduler.state_dict()}, path)

def load_ckpt(model, optimizer, scheduler, path, device='cpu'):
    ck = torch.load(path, map_location=device)
    model.load_state_dict(ck['model'])
    optimizer.load_state_dict(ck['optim'])
    scheduler.load_state_dict(ck['sched'])
    return ck['epoch'], ck['loss']

# Handle partial loads (fine-tuning with different head)
msg = model2.load_state_dict(pretrained_weights, strict=False)
print(msg.missing_keys)     # in model, not in file
print(msg.unexpected_keys)  # in file, not in model
```

Question 32

How do you export a PyTorch model to TorchScript?

TorchScript compiles a model to a graph that can run without a Python interpreter. `torch.jit.trace` records operations for a specific input — fast but misses dynamic control flow. `torch.jit.script` compiles the Python source and handles `if`/`for` branches correctly.

Python example

```python
import torch, torch.nn as nn

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(64, 10)
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return torch.relu(self.fc(x))

model = Net()

# Method 1: trace (simple models, no control flow)
traced   = torch.jit.trace(model, torch.randn(4, 64))
traced.save('model_traced.pt')

# Method 2: script (handles if/while branches)
scripted = torch.jit.script(model)
scripted.save('model_scripted.pt')

# Load without Python
loaded = torch.jit.load('model_scripted.pt')
loaded.eval()
print(loaded(torch.randn(4, 64)).shape)   # (4, 10)

# Dynamic control flow -- must use script, not trace
class DynNet(nn.Module):
    def forward(self, x: torch.Tensor,
                use_relu: bool = True) -> torch.Tensor:
        return torch.relu(x) if use_relu else torch.sigmoid(x)

scripted_dyn = torch.jit.script(DynNet())
```

Question 33

How do you export a model to ONNX and run it with ONNX Runtime?

ONNX (Open Neural Network Exchange) is a cross-framework format that enables deployment via ONNX Runtime on CPU, CUDA, TensorRT, and many other backends. Set `dynamic_axes` to allow variable batch sizes at inference time.

Python example

```python
import torch, torch.nn as nn
import onnx, onnxruntime as ort, numpy as np

model = nn.Linear(64, 10).eval()
dummy = torch.randn(1, 64)

torch.onnx.export(model, dummy, 'model.onnx',
    export_params=True, opset_version=17,
    input_names=['input'], output_names=['output'],
    dynamic_axes={'input': {0: 'batch'}, 'output': {0: 'batch'}})

onnx.checker.check_model(onnx.load('model.onnx'))
print("ONNX model valid")

# Run with ONNX Runtime
sess = ort.InferenceSession('model.onnx',
    providers=['CUDAExecutionProvider', 'CPUExecutionProvider'])

x_np = np.random.randn(4, 64).astype(np.float32)
out  = sess.run(None, {'input': x_np})[0]
print(out.shape)   # (4, 10)

# Verify outputs match PyTorch
with torch.no_grad():
    pt_out = model(torch.tensor(x_np)).numpy()
np.testing.assert_allclose(pt_out, out, atol=1e-5)
print("Outputs match!")
```

Question 34

What is quantization in PyTorch and what are the different approaches?

Quantization reduces model precision from float32 to int8 (or lower), cutting model size by $\sim 4\times$ and accelerating CPU inference. **Dynamic** quantization is the easiest (weights quantized at load time, activations at runtime). **Post-training static quantization (PTQ)** calibrates activation ranges on representative data. **Quantization-aware training (QAT)** inserts fake-quantization nodes during training for the best quality.

Python example

```python
import torch, torch.nn as nn
import torch.ao.quantization as Q

model = nn.Sequential(nn.Linear(64,32), nn.ReLU(),
                      nn.Linear(32,10)).eval()

# 1. Dynamic quantization (easiest, CPU only)
dyn = torch.quantization.quantize_dynamic(
    model, {nn.Linear}, dtype=torch.qint8)

# 2. Post-training static quantization (PTQ)
model.qconfig = Q.get_default_qconfig('x86')
Q.prepare(model, inplace=True)       # insert observers
with torch.no_grad():
    for x, _ in calibration_loader:
        model(x)                      # calibrate
Q.convert(model, inplace=True)       # replace with int8

# 3. Quantization-aware training (QAT -- best quality)
model.train()
model.qconfig = Q.get_default_qat_qconfig('x86')
Q.prepare_qat(model, inplace=True)   # fake-quant nodes
for x, y in train_loader:
    loss = criterion(model(x), y)
    loss.backward(); optimizer.step()
model.eval()
Q.convert(model, inplace=True)       # finalise int8
```

Question 35

How do you implement early stopping and model checkpointing?

Early stopping monitors a validation metric and halts training when it stops improving for `patience` consecutive epochs, saving the best checkpoint automatically. This prevents overfitting and unnecessary compute.

Python example

```python
import torch, numpy as np

class EarlyStopping:
    def __init__(self, patience=7, min_delta=1e-4,
                 mode='min', path='best.pt'):
        self.patience  = patience
        self.min_delta = min_delta
        self.mode, self.path = mode, path
        self.counter   = 0
        self.best      = np.inf if mode == 'min' else -np.inf
        self.stop      = False

    def __call__(self, metric, model):
        improved = (
            (self.mode == 'min' and metric < self.best - self.min_delta) or
            (self.mode == 'max' and metric > self.best + self.min_delta)
        )
        if improved:
            self.best = metric; self.counter = 0
            torch.save(model.state_dict(), self.path)
        else:
            self.counter += 1
            if self.counter >= self.patience:
                self.stop = True

es = EarlyStopping(patience=7, mode='min')
for epoch in range(100):
    _, val_loss = evaluate(model, val_loader, criterion, device)
    es(val_loss, model)
    if es.stop:
        print("Early stopping triggered.")
        break

model.load_state_dict(torch.load('best.pt'))
```

Chapter 8 Advanced Architectures: CNN, RNN, Transformers

Architecture questions are usually a test of depth. It is one thing to use a built-in layer. It is another to explain why it works, what tensor shapes flow through it, and how you would rebuild the core idea from scratch. This chapter keeps the focus on that second level of understanding.

Core ideas. Residual connections, convolutional blocks, recurrent layers, attention, normalization choices, and positional encodings.

What interviewers are really checking. Whether you can turn a model idea into clear PyTorch code and explain why the layers are arranged that way.

Common weak spots. Losing track of tensor shapes through the block, using the right layer for the wrong objective, and giving paper-level explanations without code-level detail.

Study advice. Sketch the tensor shapes for each block by hand. In practice, that matters more than memorizing every constructor argument.

Question 36

Implement a ResNet residual block from scratch in PyTorch.

He et al. (2016) introduced the identity shortcut $\mathbf{y} = F(\mathbf{x}, \{W_i\}) + \mathbf{x}$, where F is a 2-layer conv stack. When dimensions change (stride or channel mismatch), a projection shortcut 1×1 conv aligns shapes.

Python example

```python
import torch, torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride,
                               padding=1, bias=False)
        self.bn1   = nn.BatchNorm2d(out_ch)
        self.relu  = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False)
        self.bn2   = nn.BatchNorm2d(out_ch)

        self.shortcut = nn.Identity()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride=stride, bias=False),
                nn.BatchNorm2d(out_ch))

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return self.relu(out + identity)   # residual addition

block = ResidualBlock(64, 128, stride=2)
x     = torch.randn(4, 64, 32, 32)
print(block(x).shape)               # (4, 128, 16, 16)
```

Question 37

Implement a multi-head self-attention layer from scratch.

Vaswani et al. (2017). Project input to Q, K, V, split into h heads, compute scaled dot-product attention per head:

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

then concatenate heads and project back.

Python example

```python
import torch, torch.nn as nn, math

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        assert d_model % n_heads == 0
        self.h  = n_heads
        self.dk = d_model // n_heads
        self.q_proj = nn.Linear(d_model, d_model, bias=False)
        self.k_proj = nn.Linear(d_model, d_model, bias=False)
        self.v_proj = nn.Linear(d_model, d_model, bias=False)
        self.o_proj = nn.Linear(d_model, d_model, bias=False)
        self.drop   = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        B, T, D = x.shape

        def split(proj, t):
            return proj(t).view(B,T,self.h,self.dk).transpose(1,2)

        Q, K, V = split(self.q_proj,x), split(self.k_proj,x), split(self.v_proj,x)
        scores  = (Q @ K.transpose(-2,-1)) / math.sqrt(self.dk)
        if mask is not None:
            scores = scores.masked_fill(mask==0, float('-inf'))
        attn = self.drop(torch.softmax(scores, dim=-1))
        out  = (attn @ V).transpose(1,2).contiguous().view(B,T,D)
        return self.o_proj(out)

mhsa = MultiHeadSelfAttention(512, 8)
x    = torch.randn(2, 32, 512)
print(mhsa(x).shape)     # (2, 32, 512)
```

Question 38

How do you implement an LSTM and when would you use it vs a Transformer?

	LSTM	Transformer
Long-range deps	Weak	Strong
Parallelism	None (sequential)	Full
Memory	$O(n)$	$O(n^2)$
Streaming/causal	Natural	Needs causal mask
Short sequences	Fine	Overkill

Python example

```python
import torch, torch.nn as nn
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

class LSTMClassifier(nn.Module):
    def __init__(self, vocab, emb, hid, layers, n_cls, drop=0.3):
        super().__init__()
        self.embed = nn.Embedding(vocab, emb, padding_idx=0)
        self.lstm  = nn.LSTM(emb, hid, layers, batch_first=True,
                             dropout=drop if layers>1 else 0,
                             bidirectional=True)
        self.fc    = nn.Linear(hid*2, n_cls)
        self.drop  = nn.Dropout(drop)

    def forward(self, tokens, lengths):
        x   = self.drop(self.embed(tokens))
        pkg = pack_padded_sequence(x, lengths.cpu(), batch_first=True,
                                   enforce_sorted=False)
        _, (h_n, _) = self.lstm(pkg)
        h = torch.cat([h_n[-2], h_n[-1]], dim=-1)   # bidir last layer
        return self.fc(self.drop(h))
```

Question 39

What is the difference between Batch Normalization and Layer Normalization?

BatchNorm normalises over the batch and spatial dimensions — it requires batch size > 1 at training time and uses running statistics at eval. **LayerNorm** normalises over the feature dimension within each sample — works with any batch size, making it the standard for Transformers and sequence models.

Python example

```python
import torch, torch.nn as nn

bn   = nn.BatchNorm2d(64)    # (N,C,H,W) -- normalise over N,H,W
ln   = nn.LayerNorm(512)     # (N,T,D)   -- normalise over D
gn   = nn.GroupNorm(8, 64)   # between BN and LN

# RMSNorm (LLaMA, Mistral) -- no mean subtraction, faster
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-8):
        super().__init__()
        self.scale = nn.Parameter(torch.ones(dim))
        self.eps   = eps
    def forward(self, x):
        rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).sqrt()
        return (x / rms) * self.scale

# Usage
x  = torch.randn(2, 10, 512)   # (batch, seq, d_model)
print(ln(x).shape)              # (2, 10, 512) -- same shape
rms = RMSNorm(512)
print(rms(x).shape)             # (2, 10, 512)
```

Question 40

Implement positional encoding for a Transformer.

Three main flavours: (1) **Sinusoidal** (Vaswani 2017) — fixed, generalises to longer sequences; (2) **Learned** (GPT-style) — simple Embedding; (3) **RoPE** (Su 2021) — encodes position as rotation in the complex plane, used in LLaMA and Mistral.

Python example

```python
import torch, torch.nn as nn, math

class SinusoidalPE(nn.Module):
    def __init__(self, d_model, max_len=5000, dropout=0.1):
        super().__init__()
        self.drop = nn.Dropout(dropout)
        pe  = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float()
                        * -(math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))  # (1,L,D)

    def forward(self, x):
        return self.drop(x + self.pe[:, :x.size(1)])

# Learned PE (GPT-style)
class LearnedPE(nn.Module):
    def __init__(self, d_model, max_len=2048):
        super().__init__()
        self.embed = nn.Embedding(max_len, d_model)
    def forward(self, x):
        B, T, _ = x.shape
        pos = torch.arange(T, device=x.device).unsqueeze(0)
        return x + self.embed(pos)

# RoPE (LLaMA-style) -- rotates Q and K in frequency space
def apply_rope(q, k, seq_len, head_dim, base=10000):
    theta = 1.0 / (base ** (torch.arange(0, head_dim, 2,
                            device=q.device).float() / head_dim))
    pos   = torch.arange(seq_len, device=q.device).float()
    freqs = torch.outer(pos, theta)
    cos, sin = freqs.cos()[None,None], freqs.sin()[None,None]
    q_rot = torch.cat([-q[...,1::2], q[...,0::2]], dim=-1)
    k_rot = torch.cat([-k[...,1::2], k[...,0::2]], dim=-1)
    return q*cos + q_rot*sin, k*cos + k_rot*sin
```

Chapter 9 Distributed Training & Performance

Distributed training sounds abstract until you have to make it work across multiple GPUs or nodes. This chapter keeps the discussion practical. It compares the main parallel training patterns, explains where communication costs show up, and highlights the setup details that often break otherwise correct code.

Core ideas. Data parallelism, DistributedDataParallel, Fully Sharded Data Parallel, sharding, tensor and pipeline parallel ideas, and launch patterns for multi-process jobs.

What interviewers are really checking. Whether you know when a scaling strategy helps, what cost it introduces, and why newer distributed patterns usually replace older wrappers.

Common weak spots. Ignoring sampler coordination, misunderstanding when gradients synchronize, and treating all multi-GPU approaches as basically the same thing.

Study advice. Compare each method in terms of memory, communication overhead, and operational complexity. That comparison is often what interviewers want to hear.

Question 41

What is the difference between `nn.DataParallel` and `DistributedDataParallel`?

	DataParallel	DDP
Processes	1	1 per GPU
Grad sync	Parameter server (GPU 0)	All-reduce (NCCL)
Multi-node	No	Yes
GIL bottleneck	Yes	No
Recommended	No	Yes

Python example

```python
import torch, torch.nn as nn, torch.distributed as dist, os
from torch.nn.parallel import DistributedDataParallel as DDP

def setup(rank, world_size):
    dist.init_process_group(backend='nccl', init_method='env://',
                            rank=rank, world_size=world_size)
    torch.cuda.set_device(rank)

def train(rank, world_size):
    setup(rank, world_size)
    model    = nn.Linear(1024, 10).to(rank)
    ddp_model = DDP(model, device_ids=[rank])
    optimizer = torch.optim.Adam(ddp_model.parameters())

    for x, y in distributed_loader:
        x, y = x.to(rank), y.to(rank)
        loss  = criterion(ddp_model(x), y)
        loss.backward()    # all-reduce happens here
        optimizer.step(); optimizer.zero_grad()

    dist.destroy_process_group()

# Launch: torchrun --nproc_per_node=4 train.py
```

Question 42

How does `FullyShardedDataParallel` (FSDP) enable training of very large models?

FSDP shards parameters, gradients, *and* optimizer states across all GPUs, so each GPU holds only $1/N$ of the total memory. Parameters are all-gathered before each forward/backward layer and immediately discarded after, enabling models that exceed a single GPU's VRAM.

Python example

```python
import torch, torch.nn as nn, functools
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP, MixedPrecision,
    BackwardPrefetch, ShardingStrategy)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from transformers.models.llama.modeling_llama import LlamaDecoderLayer

mp = MixedPrecision(param_dtype=torch.bfloat16,
                    reduce_dtype=torch.float32)
wrap = functools.partial(transformer_auto_wrap_policy,
                         transformer_layer_cls={LlamaDecoderLayer})

model = FSDP(model,
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    mixed_precision=mp, auto_wrap_policy=wrap,
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
    device_id=torch.cuda.current_device())

# Save with FSDP: consolidate shards on rank 0
from torch.distributed.fsdp import StateDictType, FullStateDictConfig
cfg = FullStateDictConfig(offload_to_cpu=True, rank0_only=True)
with FSDP.state_dict_type(model, StateDictType.FULL_STATE_DICT, cfg):
    state = model.state_dict()
    if dist.get_rank() == 0:
        torch.save(state, 'model.pt')
```

Question 43

How do you implement a `DistributedSampler` for DDP training?

`DistributedSampler` partitions the dataset across ranks so each GPU sees a disjoint subset of samples. Call `sampler.set_epoch(epoch)` at the start of every epoch to re-shuffle with a different random seed — skipping this means all epochs see the same order.

Python example

```python
from torch.utils.data import DataLoader
from torch.utils.data.distributed import DistributedSampler

sampler = DistributedSampler(
    dataset, num_replicas=world_size, rank=rank,
    shuffle=True, drop_last=True)

loader = DataLoader(dataset, batch_size=64, sampler=sampler,
                    num_workers=4, pin_memory=True,
                    persistent_workers=True)

# CRITICAL: set epoch each time for correct shuffling
for epoch in range(num_epochs):
    sampler.set_epoch(epoch)   # changes random seed per epoch
    for x, y in loader:
        ...

# Without set_epoch: every epoch uses the same shuffle
# -> all ranks see the same ordering -> wrong!
```

Question 44

What is pipeline parallelism and how is it different from tensor parallelism?

Pipeline parallelism splits model layers across GPUs; data flows through stages like an assembly line with micro-batching to hide pipeline bubbles. **Tensor parallelism** (Megatron-LM) splits individual weight matrices across GPUs and uses all-reduce per layer. Large-scale LLM training combines both together with DDP — known as 3D parallelism.

Python example

```python
import torch, torch.nn as nn

# Pipeline parallelism: each GPU holds a subset of layers
stage0 = nn.Sequential(embedding, layer0, layer1).to('cuda:0')
stage1 = nn.Sequential(layer2, layer3, head).to('cuda:1')

def forward_pipeline(x):
    x = stage0(x)                       # on GPU 0
    x = x.to('cuda:1')                  # explicit transfer
    return stage1(x)                    # on GPU 1

# Tensor parallelism: split weight matrix column-wise
class ColParallelLinear(nn.Module):
    def __init__(self, in_f, out_f, world_size):
        super().__init__()
        self.linear = nn.Linear(in_f, out_f // world_size)

    def forward(self, x):
        local = self.linear(x)
        out   = [torch.zeros_like(local)
                 for _ in range(dist.get_world_size())]
        dist.all_gather(out, local)
        return torch.cat(out, dim=-1)   # reconstructed output
```

Question 45

How do you use `torchrun` and configure a distributed training job?

`torchrun` (replacing the old `torch.distributed.launch`) automatically sets the environment variables `RANK`, `LOCAL_RANK`, and `WORLD_SIZE` and manages worker fault tolerance. Your script reads those variables to initialise the process group.

Python example

```python
# train_ddp.py
import os, torch, torch.distributed as dist, torch.nn as nn
from torch.nn.parallel import DistributedDataParallel as DDP

def main():
    rank       = int(os.environ['RANK'])
    local_rank = int(os.environ['LOCAL_RANK'])
    world_size = int(os.environ['WORLD_SIZE'])

    dist.init_process_group(backend='nccl')
    torch.cuda.set_device(local_rank)

    model = nn.Linear(512, 10).to(local_rank)
    model = DDP(model, device_ids=[local_rank])
    # ... training loop ...
    dist.destroy_process_group()

if __name__ == '__main__':
    main()
```

Bash

```bash
# Single node, 4 GPUs
torchrun --nproc_per_node=4 train_ddp.py

# Multi-node (2 nodes, 8 GPUs each)
# On node 0 (master):
torchrun --nproc_per_node=8 --nnodes=2 --node_rank=0 \
  --master_addr="192.168.1.10" --master_port=29500 train_ddp.py
# On node 1:
torchrun --nproc_per_node=8 --nnodes=2 --node_rank=1 \
  --master_addr="192.168.1.10" --master_port=29500 train_ddp.py
```

Chapter 10 Debugging, Profiling & Best Practices

This final chapter is about engineering judgement. Most real PyTorch pain comes from debugging, not from writing the first version of the model. When a run diverges, gradients explode, or memory grows every epoch, you need a method, not guesswork. These questions give you a chance to show that method clearly.

Core ideas. Gradient-path debugging, NaN tracing, reproducibility settings, targeted unit tests, profiler usage, and memory-leak diagnosis.

What interviewers are really checking. Whether you troubleshoot in a disciplined order and can separate symptoms from root causes without guessing.

Common weak spots. Changing too many variables at once, assuming the optimizer is always at fault, and skipping basic checks such as input statistics, shapes, or deterministic settings.

Study advice. Treat this chapter like an incident-response playbook. The best answers are calm, stepwise, and grounded in observable signals.

Question 46

How do you diagnose and fix vanishing/exploding gradients?

Log gradient norms per layer at regular intervals. Norms below 10^{-5} signal vanishing; above 10^{3} signal exploding. Fixes: gradient clipping, proper weight initialisation (He/Xavier), residual connections, and normalisation layers.

Python example

```python
import torch, torch.nn as nn

def log_grad_norms(model):
    for name, p in model.named_parameters():
        if p.grad is not None:
            print(f"{name}: {p.grad.norm().item():.4e}")

for step, (x, y) in enumerate(loader):
    loss = criterion(model(x), y)
    loss.backward()
    if step % 100 == 0: log_grad_norms(model)
    nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # fix exploding
    optimizer.step(); optimizer.zero_grad()

# Fix: proper initialisation
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
        if m.bias is not None: nn.init.zeros_(m.bias)
    elif isinstance(m, nn.Conv2d):
        nn.init.kaiming_normal_(m.weight, mode='fan_out')

model.apply(init_weights)

# Fix: LayerNorm in transformers, BatchNorm in CNNs
# Fix: ReLU/GELU activations -- avoid Sigmoid/Tanh in deep nets
```

Question 47

How do you track NaN values during training and identify their source?

Use `torch.autograd.set_detect_anomaly(True)` to get a full stack trace when a NaN gradient is produced. Register forward hooks to pinpoint the exact layer. Common causes: `log(0)`, `0/0`, unscaled fp16 loss, and learning rates that are too large.

Python example

```python
import torch, torch.nn as nn

# Detect NaN in loss immediately
for step, (x, y) in enumerate(loader):
    out  = model(x)
    loss = criterion(out, y)
    if torch.isnan(loss) or torch.isinf(loss):
        print(f"NaN at step {step}:")
        print(f"  out: min={out.min():.4f} max={out.max():.4f} "
              f"nan={out.isnan().any()}")
        raise ValueError("NaN in loss")
    loss.backward()

# anomaly detection: raises with exact stack trace (slow!)
with torch.autograd.set_detect_anomaly(True):
    model(x).sum().backward()

# Forward hook to find the offending layer
def nan_hook(module, inp, out):
    if isinstance(out, torch.Tensor) and out.isnan().any():
        print(f"NaN in: {module.__class__.__name__}")

for mod in model.modules():
    mod.register_forward_hook(nan_hook)

# Common fixes:
# log(0)  -> add eps: torch.log(x.clamp(min=1e-8))
# 0/0     -> guard in custom ops
# fp16    -> use GradScaler, or switch to bfloat16
```

Question 48

How do you make PyTorch training fully reproducible?

Reproducibility requires seeding Python, NumPy, and PyTorch (CPU and all GPUs); disabling non-deterministic cuDNN algorithms; and seeding each DataLoader worker independently. Note: `deterministic=True` can slow training by 10–30%.

Python example

```python
import torch, numpy as np, random, os

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark     = False
    os.environ['CUBLAS_WORKSPACE_CONFIG'] = ':4096:8'
    torch.use_deterministic_algorithms(True)

set_seed(42)

# DataLoader worker seeding
def worker_init_fn(worker_id):
    base = torch.initial_seed() % 2**32
    np.random.seed(base + worker_id)
    random.seed(base + worker_id)

loader = DataLoader(
    dataset, num_workers=4,
    worker_init_fn=worker_init_fn,
    generator=torch.Generator().manual_seed(42),
)
```

Question 49

How do you write unit tests for PyTorch neural network components?

Good neural network tests check: (1) output shapes; (2) gradient flow; (3) no NaN outputs; (4) numerical gradient correctness via `gradcheck`. Run these in CI on every model change — they catch regressions in custom layers immediately.

Python example

```python
import torch, unittest

class TestMHSA(unittest.TestCase):
    def setUp(self):
        torch.manual_seed(0)
        self.model = MultiHeadSelfAttention(d_model=64, n_heads=4)
        self.B, self.T, self.D = 2, 10, 64

    def test_output_shape(self):
        x   = torch.randn(self.B, self.T, self.D)
        out = self.model(x)
        self.assertEqual(out.shape, (self.B, self.T, self.D))

    def test_gradients_flow(self):
        x   = torch.randn(self.B, self.T, self.D, requires_grad=True)
        out = self.model(x)
        out.sum().backward()
        self.assertIsNotNone(x.grad)
        self.assertFalse(x.grad.isnan().any())

    def test_no_nan_output(self):
        x   = torch.randn(self.B, self.T, self.D)
        out = self.model(x)
        self.assertFalse(out.isnan().any())
        self.assertFalse(out.isinf().any())

    def test_gradcheck(self):
        m  = MultiHeadSelfAttention(64, 4).double()
        x  = torch.randn(1, 4, 64, dtype=torch.float64,
                         requires_grad=True)
        self.assertTrue(torch.autograd.gradcheck(m, (x,), atol=1e-3))

if __name__ == '__main__':
    unittest.main()
```

Question 50

What are common memory leaks in PyTorch training loops and how do you find them?

The most common leak is appending live tensor objects (which hold the computation graph) to a Python list. Extracting `.item()` gives you the scalar value with no graph attached. Monitor `memory_allocated()` across epochs — steadily rising values with no model size change indicate a reference leak.

Python example

```python
import torch, gc

# WRONG: graph lives in the list across the entire loop
losses = []
for x, y in loader:
    loss = model(x).sum()
    losses.append(loss)           # holds graph in memory!

# RIGHT: extract scalar -- no graph reference
losses = []
for x, y in loader:
    loss = model(x).sum()
    losses.append(loss.item())   # Python float, no graph

# Monitor memory across epochs
def check_mem(tag=""):
    alloc = torch.cuda.memory_allocated() / 1e9
    res   = torch.cuda.memory_reserved() / 1e9
    print(f"[{tag}] Allocated={alloc:.3f}GB Reserved={res:.3f}GB")

for epoch in range(10):
    check_mem(f"Epoch {epoch} start")
    for x, y in loader:
        loss = criterion(model(x), y)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad(set_to_none=True)  # free grad memory
    check_mem(f"Epoch {epoch} end")
    gc.collect(); torch.cuda.empty_cache()
```

Quick-reference study summary

Use this section for a fast final review. Each row is a prompt you should be able to explain in one or two clear minutes, with the key concept acting as your memory anchor.

Q	Question	Category	Key concept
1	Tensor vs NumPy	Fundamentals	GPU, autograd, zero-copy
2	Tensor creation	Fundamentals	zeros, rand, arange
3	Reshaping	Fundamentals	view, reshape, squeeze
4	Broadcasting	Fundamentals	Shape alignment, silent bugs
5	einsum	Fundamentals	Tensor contractions
6	Autograd internals	Autograd	`grad_fn`, reverse-mode AD
7	`no_grad` / detach	Autograd	Inference, target networks
8	In-place ops	Autograd	Version counter, graph corruption
9	Custom Function	Autograd	STE, forward/backward
10	Gradient accumulation	Autograd	Large batch simulation
11	Parameter vs Buffer	`nn.Module`	`state_dict`, device movement
12	Hooks	`nn.Module`	Feature extraction, grad surgery
13	Freezing params	`nn.Module`	Transfer learning
14	Weight sharing	`nn.Module`	Tied autoencoders
15	ModuleList / Dict / Seq	`nn.Module`	Container differences
16	Training loop	Training	Production loop, AMP
17	Optimizers	Training	SGD, Adam, AdamW
18	LR schedulers	Training	Step, Cosine, OneCycle
19	Loss functions	Training	CE, BCE, Huber, Focal
20	Mixed precision	Training	AMP, fp16, bf16
21	Custom Dataset	DataLoader	`__getitem__`, `__len__`
22	IterableDataset	DataLoader	Streaming, worker sharding
23	`collate_fn`	DataLoader	Variable-length sequences
24	Loader profiling	DataLoader	Throughput, GPU augmentation
25	Class imbalance	DataLoader	WeightedSampler, Focal
26	GPU memory / OOM	CUDA	Caching allocator, debugging
27	`pin_memory`	CUDA	DMA, non-blocking transfer
28	CUDA streams	CUDA	Overlap compute and I/O
29	`torch.compile`	CUDA	TorchDynamo, Inductor
30	Profiling	CUDA	Profiler, CUDA events
31	`state_dict` save	Deployment	Portable checkpointing
32	TorchScript	Deployment	trace vs script
33	ONNX export	Deployment	Dynamic axes, ORT
34	Quantization	Deployment	Dynamic, PTQ, QAT
35	Early stopping	Deployment	Patience, best checkpoint

Continued on next page

Q	Question	Category	Key concept
36	ResNet block	Architectures	Residual connections
37	Multi-head attention	Architectures	QKV, scaled dot-product
38	LSTM	Architectures	pack/pad, bidirectional
39	BatchNorm vs LayerNorm	Architectures	Normalisation axis
40	Positional encoding	Architectures	Sinusoidal, learned, RoPE
41	DP vs DDP	Distributed	All-reduce, one process/GPU
42	FSDP	Distributed	Sharding, large models
43	DistributedSampler	Distributed	`set_epoch`, no duplicates
44	Pipeline / tensor parallel	Distributed	3D parallelism
45	`torchrun`	Distributed	Multi-node launch
46	Vanishing / exploding	Debugging	Grad norms, clipping
47	NaN detection	Debugging	Anomaly detection, hooks
48	Reproducibility	Debugging	Seeds, deterministic CUDA
49	Unit testing	Debugging	Shape, grad, `gradcheck`
50	Memory leaks	Debugging	`.item()`, `zero_grad`, gc

Prepared by Lamhot Siagian | AI Engineering Insider